NORTH SOUTH UNIVERSITY
Department of Electrical and Computer Engineering

OPERATING SYSTEMS PROJECT REPORT

DEADLOCK DETECTION & RECOVERY TOOL

Resource Allocation Graph • Deadlock Detection • Recovery • SDK

Project Information	Details
Student	Farhana Akther Layba Mitu
Student ID	1911184042
Course Code	CSE323
Section	10
Faculty / Initial	RMz1
Supervisor	Dr. Rashed Mazumder

Submission Date: 02.09.2026

Declaration

I, Farhana Akther Layba Mitu (ID: 1911184042), hereby declare that this project report titled "Deadlock Detection & Recovery Tool", submitted for the course CSE323 (Section 10) at North South University, is the result of my own work carried out under the guidance and supervision of Dr. Rashed Mazumder (Faculty Initial: RMz1).

I further declare that the design, implementation, testing, and documentation presented in this report — including the Resource Allocation Graph model, the DFS-based deadlock detection algorithm, the recovery mechanisms, the reusable SDK layer, and the graphical interface — were developed by me, and that this work has not been submitted, in whole or in part, for any other course, degree, or qualification at this or any other institution.

Any external material referenced in this report, including published papers and online documentation, has been properly cited in the References section.

_____

Farhana Akther Layba Mitu
ID: 1911184042
CSE323, Section 10

Abstract

Deadlock is a fundamental problem in operating systems in which a group of processes becomes permanently blocked because each process is waiting for a resource held by another process within the same circular chain of dependencies. This project presents the Deadlock Detection & Recovery Tool, a Python-based desktop application that models processes and resources using a Resource Allocation Graph (RAG), detects circular-wait conditions using a Depth-First Search (DFS) based algorithm, and resolves detected deadlocks through three recovery strategies: process termination, resource preemption, and cost-aware lowest-cost recovery.

The system is built with a layered architecture that separates the graphical interface (Tkinter) from the reusable core logic through a dedicated SDK layer (DeadlockSDK), allowing the detection and recovery engine to be reused independently of the GUI. The default demonstration scenario models five processes (P1–P5) and ten resources (R1–R10), with a deliberately constructed ten-edge cycle that the DFS detector correctly identifies and reports. Recovery actions are verified by re-running detection immediately afterward, confirming whether the cycle has been broken.

The tool serves as both an educational aid for visualizing deadlock concepts taught in operating-systems courses and a small, reusable reference implementation that other projects can build upon. Functional testing across graph construction, detection, all three recovery methods, system reset, SDK-level access, and GUI integration confirmed that the system behaves as designed.

Keywords

Deadlock, Operating System, Resource Allocation Graph, Depth-First Search, Cycle Detection, Process Termination, Resource Preemption, Lowest-Cost Recovery, Tkinter, NetworkX, Matplotlib, Software Development Kit (SDK).

1. Introduction

Deadlock is one of the classic problems studied in operating systems, occurring when a group of processes are each waiting for a resource that is held by another process in the same group, so that none of them can ever proceed. As operating systems and multi-process applications grow more concurrent, understanding, visualizing, and resolving deadlocks becomes essential for both education and system design. Most textbook treatments of deadlock stop at theory — the Resource Allocation Graph (RAG), cycle detection, and recovery strategies are usually explained on paper without an interactive tool that lets a learner build a graph, watch a cycle form, and apply a recovery method to see the system return to a safe state.

The Deadlock Detection & Recovery Tool is a desktop application built to close that gap. It allows a user to construct a Resource Allocation Graph consisting of processes and resources, run a Depth-First Search (DFS) based cycle-detection algorithm on that graph, and then resolve any detected deadlock using either process termination, resource preemption, or an automatic lowest-cost recovery strategy. The tool is implemented in Python using Tkinter for the graphical interface and NetworkX/Matplotlib for graph construction and visualization. In addition to the GUI, the project exposes a reusable Deadlock SDK — a clean, GUI-independent API layer that lets the same detection and recovery logic be reused by other applications, such as a future web dashboard or an automated monitoring service.

2. Problem Statement

Operating systems courses typically teach deadlock through static diagrams and manually solved examples. This creates several practical issues for learners and small-scale system designers:

- Deadlock is difficult to visualize mentally, especially when several processes and resources are interlinked through both request and allocation edges.
- Most available teaching material explains detection and recovery separately, without a working demonstration of the two working together on the same data.
- There is no lightweight, reusable component that a student or a small project can plug into its own system to add deadlock detection and recovery without re-implementing the graph and DFS logic from scratch.
- Recovery decisions (which process to terminate, which resource to preempt) are rarely demonstrated with a concrete, comparable cost model.

The Deadlock Detection & Recovery Tool addresses these gaps by combining a visual Resource Allocation Graph, an explicit DFS-based cycle detector, multiple recovery strategies, and a reusable SDK layer in a single, self-contained project.

3. Impact of the Problem

3.1 Educational Impact

- Students often memorize the definition of deadlock without seeing how a cycle actually forms edge-by-edge in a Resource Allocation Graph.
- Without a working detector, the DFS-based cycle detection algorithm remains an abstract pseudocode exercise rather than a concrete, testable implementation.

3.2 System Design Impact

- Systems that hard-code deadlock handling directly inside application logic are harder to maintain and cannot be reused elsewhere.
- Without a clear separation between the GUI and the core logic, the detection and recovery algorithms cannot be tested independently or integrated into another interface (web, mobile, or command line).

3.3 Operational Impact

- Choosing an arbitrary process to terminate during recovery can be more expensive than necessary if the relative termination cost of each process is not considered.
- Manually resolving a deadlock without a defined strategy (termination vs. preemption) increases the risk of repeating the same circular wait.

4. Project Goals

- Build a graphical Resource Allocation Graph (RAG) that models processes, resources, request edges, and allocation edges.
- Implement a Depth-First Search (DFS) based algorithm to detect a deadlock cycle and report the exact sequence of processes and resources involved.
- Provide multiple recovery methods: process termination, resource preemption, and an automatic lowest-cost recovery strategy.
- Visualize the graph before and after recovery so the effect of each action is immediately visible.
- Expose the core detection and recovery functionality through a reusable SDK layer (DeadlockSDK), independent of the GUI.
- Provide a clean, professional Tkinter GUI so the whole detect-and-recover cycle can be demonstrated without using the command line.

- Keep the codebase modular (graph, detection, recovery, SDK, GUI as separate files) so each part can be tested, extended, or reused independently.

5. System Overview

At its core, the system maintains a single Resource Allocation Graph representing five processes (P1–P5) and ten resources (R1–R10). Every process that is waiting for a resource is connected to that resource by a request edge, and every resource currently held by a process is connected back to that process by an allocation edge. The default demonstration scenario deliberately links five processes and five resources into a ten-edge cycle, while the remaining five resources are allocated without creating any further dependency, so that the resulting deadlock is clean and easy to follow.

The screenshot below, captured directly from the running application's “Show Resource Graph” feature, shows this exact graph: blue nodes are processes, orange nodes are resources, red edges are requests, and green edges are allocations.

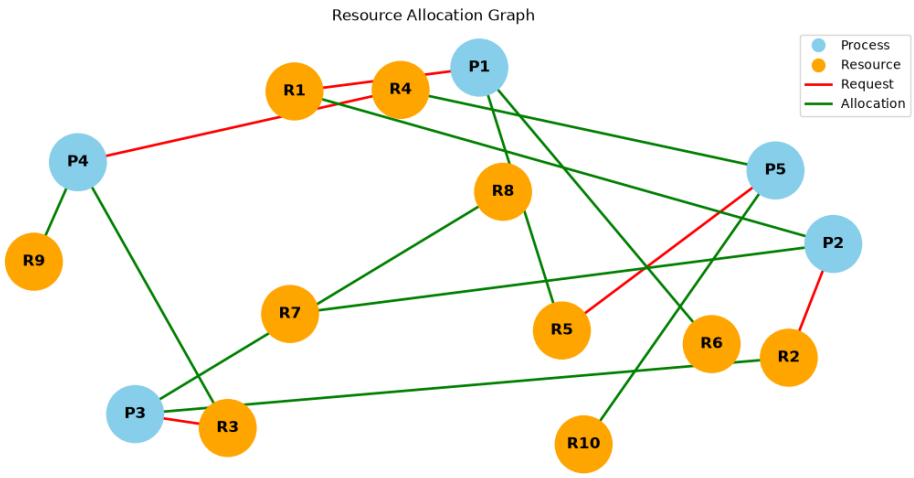

Figure 1: Resource Allocation Graph, (Show Resource Graph).

From this starting state, the application can detect the deadlock cycle, apply a recovery method, and re-verify the outcome — the complete walkthrough is presented in Section 15 (Demonstration & Expected Results).

6. Technologies Used / System Implementation

The project is implemented entirely in Python, combining a small set of well-established libraries for graph modeling, visualization, and desktop UI. The table below maps each technology to the part of the codebase that uses it.

Technology / Library	Purpose in the Project	Main Classes / Modules
Python 3	Core programming language for the entire application	All modules
Tkinter (tkinter, ttk, messagebox, simpledialog)	Graphical user interface: dashboard, buttons, dialogs, alerts	DeadlockApp (gui.py)
NetworkX	Directed graph modeling (nodes, edges, successors)	ResourceAllocationGraph (graph.py)
Matplotlib	Rendering the Resource Allocation Graph with colored nodes, edges, and legend	ResourceAllocationGraph.show_graph()
Python collections (set, list, dict)	Visited set, recursion stack, and path tracking during DFS; recovery summaries	DeadlockDetector, DeadlockRecovery

7. Target Users

Unlike a production marketplace or enterprise system, this project targets a smaller, focused set of users centered on learning and demonstrating operating-system concepts:

- Students and instructors who want an interactive way to teach or learn Resource Allocation Graphs, cycle detection, and deadlock recovery strategies.
- Developers of other small systems who want to plug in ready-made deadlock detection and recovery logic through the DeadlockSDK, instead of writing it from scratch.
- Anyone running a live demonstration of deadlock detection and recovery, using the GUI's step-by-step flow: build graph → detect → recover → verify → reset.

8. Functional Requirements

8.1 Resource Allocation Graph Construction

- The system can add processes and resources as distinct, color-coded nodes (processes in blue, resources in orange).
- The system can create request edges (process → resource) and allocation edges (resource → process).
- The system can render the graph with NetworkX/Matplotlib, showing request edges in red and allocation edges in green, with a legend.

8.2 Deadlock Detection

- The system performs a DFS traversal over the directed graph, tracking visited nodes, a recursion stack, and the current path.
- If a neighbor is already present in the recursion stack, the system reports a cycle — the point at which the neighbor first appears in the path marks the start of the deadlock cycle.
- The system reports the exact cycle (e.g., P1 → R1 → P2 → R2 → P3 → R3 → P4 → R4 → P5 → R5 → P1) and lists which processes are involved in the deadlock.
- If no cycle exists, the system reports that the graph is safe.

8.3 Deadlock Recovery

- Process Termination: the user selects a deadlocked process; the system removes that process node along with its associated request/allocation edges, releasing its resources.
- Resource Preemption: the user specifies a resource, its current holder, and the requesting process; the system removes the old allocation and request edges and reallocates the resource to the requester.
- Lowest-Cost Recovery: the system automatically selects the active process with the lowest predefined termination cost and terminates it, minimizing recovery overhead.
- After any recovery action, the system re-runs deadlock detection to confirm whether the cycle has been broken and displays a recovery summary (method used, process/resource affected, success status).

8.4 SDK Layer

- DeadlockSDK exposes RAG construction, deadlock detection, cycle retrieval, process termination, resource preemption, lowest-cost recovery, and system status/summary as simple method calls.
- The SDK can be imported and used independently of the Tkinter GUI, for example from another script or a future interface.

8.5 Graphical User Interface

- The GUI provides buttons for showing the graph, detecting deadlock, terminating a process, preempting a resource, running lowest-cost recovery, and resetting the system.
- The GUI displays live system status (e.g., "SYSTEM READY", "DEADLOCK CYCLE", "SDK CONNECTED") and recovery status.

9. Non-Functional Requirements

- Usability: A clean, color-coded GUI with clearly labeled buttons for each stage of the detect-and-recover workflow.
- Modularity: Graph construction, detection, recovery, SDK, and GUI are separated into distinct files/classes so each can be modified or reused independently.
- Reusability: The DeadlockSDK is designed to be consumed by other applications without depending on Tkinter.
- Correctness: Deadlock detection and recovery are re-verified after every action so the reported system status always reflects the current graph state.
- Maintainability: The codebase follows a consistent structure and naming convention across all modules (graph.py, deadlock_detection.py, deadlock_recovery.py, sdk.py, gui.py).
- Portability: Because the project is written in pure Python with widely available libraries (Tkinter, NetworkX, Matplotlib), it can run on any platform with a Python interpreter.

10. System Design — Layered Architecture

The system follows a layered architecture that keeps the presentation layer (GUI) separate from the reusable core logic, with the SDK acting as an integration layer between the two. This separation means the detection and recovery algorithms can be tested and reused independently of Tkinter, and a different interface (command line, web, or mobile) could reuse the same SDK without modification.

Layer	Responsibility	Files / Classes
Presentation Layer (GUI)	Provides the interactive dashboard, operation buttons, status cards, and confirmation dialogs	gui.py — DeadlockApp
Integration Layer (SDK)	Exposes a clean, reusable, GUI-independent API over the core logic	sdk.py — DeadlockSDK
Core Logic Layer	Graph modeling and visualization; DFS-based cycle detection; termination and preemption recovery	graph.py — ResourceAllocationGraph deadlock_detection.py — DeadlockDetector deadlock_recovery.py — DeadlockRecovery
Entry Point	Console-driven demonstration of the default scenario	main.py

11. Data Model — Resource Allocation Graph

Because this is a graph-based algorithmic tool rather than a database-backed application, it does not have relational tables or foreign keys, so a conventional Entity-Relationship diagram does not apply here. Instead, the equivalent structural model is the Resource Allocation Graph itself: a directed graph in which every node is either a Process or a Resource, and every edge is either a Request (Process → Resource) or an Allocation (Resource → Process).

The diagram below reproduces the exact default scenario used in this project's code (sdk.py / main.py): five processes (P1–P5) and five of the ten resources are connected through requests and allocations, forming the deadlock cycle P1 → R1 → P2 → R2 → P3 → R3 → P4 → R4 → P5 → R5 → P1.

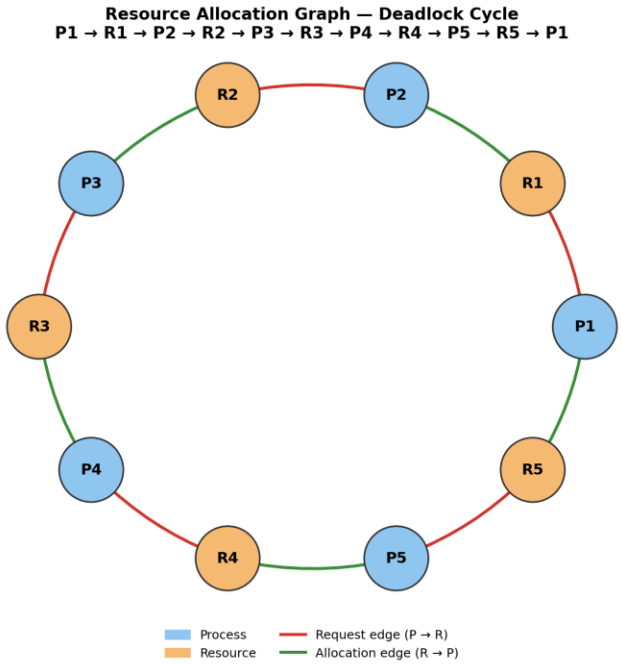

Figure 2: Resource Allocation Graph — Data Model / Deadlock Cycle

12. Algorithm and Core Logic

12.1 Deadlock Detection Using DFS

The core detection algorithm performs a Depth-First Search over the directed Resource Allocation Graph, maintaining three structures: a visited set (nodes already explored), a recursion_stack set (nodes on the current DFS path), and a path list (the current path in order). The algorithm proceeds as follows:

- Build the Resource Allocation Graph from the current processes, resources, requests, and allocations.

- Starting from each unvisited node, recursively visit its successors, adding each visited node to the recursion stack and the path.
- If a neighbor is already in the recursion stack, a back-edge has been found — this means a cycle exists. The starting point of the cycle is located using the neighbor's index in the current path.
- Once a full traversal branch completes without finding a cycle, the node is removed from the recursion stack and popped from the path (standard DFS backtracking).
- If any traversal reports a cycle, the exact sequence of processes and resources in that cycle is returned as the deadlock cycle; otherwise, the graph is reported as free of deadlock.

For the project's default scenario, DFS starting at P1 visits R1, P2, R2, P3, R3, P4, R4, and P5 in turn. When the traversal reaches R5 and looks at its successor P1, it finds P1 already in the recursion stack — this back-edge is the signal that closes the cycle P1 → R1 → P2 → R2 → P3 → R3 → P4 → R4 → P5 → R5 → P1, which the tool reports as the deadlock.

12.2 DFS Time Complexity

Let V be the total number of nodes in the Resource Allocation Graph (processes plus resources) and E be the total number of edges (requests plus allocations). Because the algorithm visits each node once and inspects each outgoing edge at most once, the overall time complexity is O(V + E), which is the standard complexity of Depth-First Search on a directed graph. The space complexity is also O(V) for the visited set, the recursion stack, and the path list.

In the default scenario, V = 15 (5 processes + 10 resources) and E = 15 (5 request edges + 10 allocation edges), so the detector examines a small, constant number of nodes and edges — in practice, the detection completes in a single pass over the graph regardless of which node the traversal starts from.

12.3 Recovery Strategies

Once a deadlock cycle has been confirmed, the system offers three recovery strategies, all implemented in deadlock_recovery.py and exposed through both the GUI and the SDK.

12.3.1 Process Termination

- A deadlocked process is selected and removed from the graph, along with all of its request and allocation edges.
- Releasing that process's resources can satisfy the waiting requests of other processes, which breaks the circular wait.
- Complexity: removing a node and its incident edges is proportional to the node's degree, i.e., O(degree(process)).

12.3.2 Resource Preemption

- A resource is temporarily taken away from the process currently holding it and reallocated to the process that was requesting it.
- This requires validating that the holder-resource allocation edge and the requester-resource request edge both exist before the swap is performed.
- The process that lost the resource is not terminated and may resume once resources become available again — useful when termination is undesirable.

12.3.3 Lowest-Cost Recovery

- Each process is associated with a predefined termination cost (P1 = 5, P2 = 4, P3 = 3, P4 = 2, P5 = 1 in the default scenario).
- The system scans all currently active processes, in O(n) time where n is the number of active processes, and selects the one with the minimum cost.
- That process is then terminated using the same process-termination routine described in Section 12.3.1, minimizing the overall cost of recovery.

Regardless of which method is used, the tool always re-runs the DFS detector immediately afterward and reports whether the cycle has actually been broken, following the general recommendation that a detection algorithm should be invoked again after each recovery step.

13. Class Diagram

The class diagram below reflects the actual module and class structure of the codebase. ResourceAllocationGraph is the shared data structure used by every other module; DeadlockDetector and DeadlockRecovery each operate directly on that graph; DeadlockSDK creates and owns its own graph instance and composes the detector and recovery classes into a single reusable API; and both the GUI (DeadlockApp) and the console demo (main.py) sit on top of these core classes.

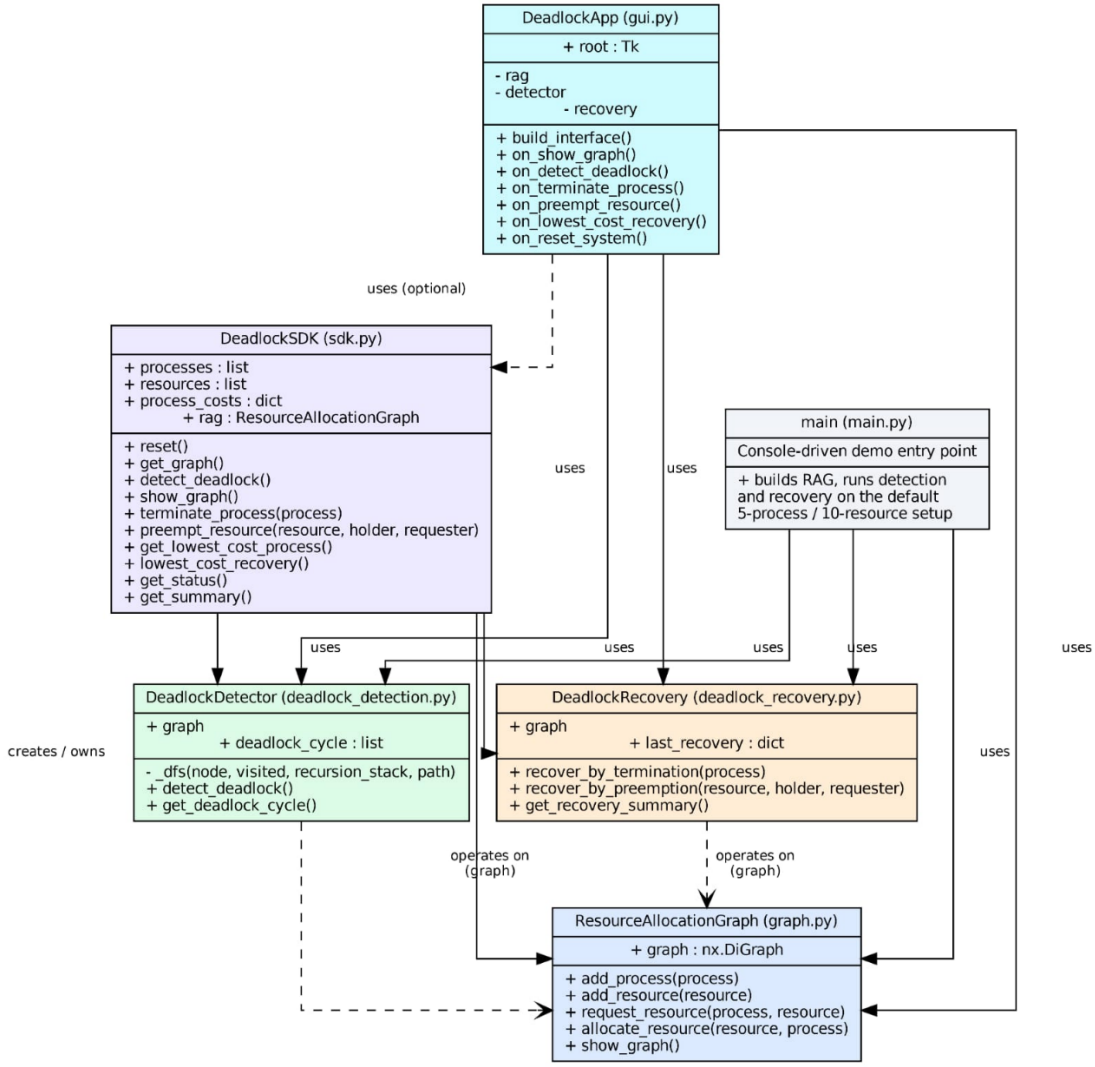

Figure 3: Class Diagram of the Deadlock Detection & Recovery Tool.

14. SDK Integration

The Deadlock SDK (DeadlockSDK) is a reusable interface layer that exposes the core deadlock functionality without requiring direct access to the GUI. It sits between the GUI and the core modules (Resource Allocation Graph, DFS Detector, and Recovery engine) and provides the following reusable operations:

- Build and manage the Resource Allocation Graph
- Detect a deadlock cycle
- Retrieve exact cycle information
- Terminate a process
- Preempt a resource

- Run lowest-cost recovery
- Report system status and a full summary (total/active processes, total resources, deadlock state, cycle, status)

The SDK is useful because it separates presentation from core logic, makes the detection and recovery functionality reusable outside of Tkinter, provides a clean API-style interface, and makes it easier to integrate the same engine into another application later — such as a web or mobile client, or an external monitoring service.

14.1 Example of SDK Usage

The snippet below shows how another script — with no Tkinter dependency at all — can drive the same detection and recovery engine used by the GUI:

```
from sdk import DeadlockSDK

sdk = DeadlockSDK()                 # builds the default RAG scenario

print(sdk.get_status())             # "DEADLOCK DETECTED" or "SYSTEM SAFE"

found, cycle = sdk.detect_deadlock()
if found:
    print("Cycle:", " -> ".join(cycle))

# Resolve automatically using the lowest-cost strategy
success, message = sdk.lowest_cost_recovery()
print(message)                      # e.g. "P5 terminated using lowest-cost recovery.
Cost = 1"

print(sdk.get_summary())            # totals, active processes, deadlock flag, cycle,
status
```

This is exactly the separation of concerns described in Section 10: the SDK makes the deadlock engine reusable instead of tying it permanently to the GUI.

15. Demonstration & Expected Results

This section walks through a full detect-and-recover session using screenshots captured directly from the running application, in the order they were produced. Every screenshot below is real output from the tool described in this report — not a mock-up.

Step 1 — Deadlock Detected

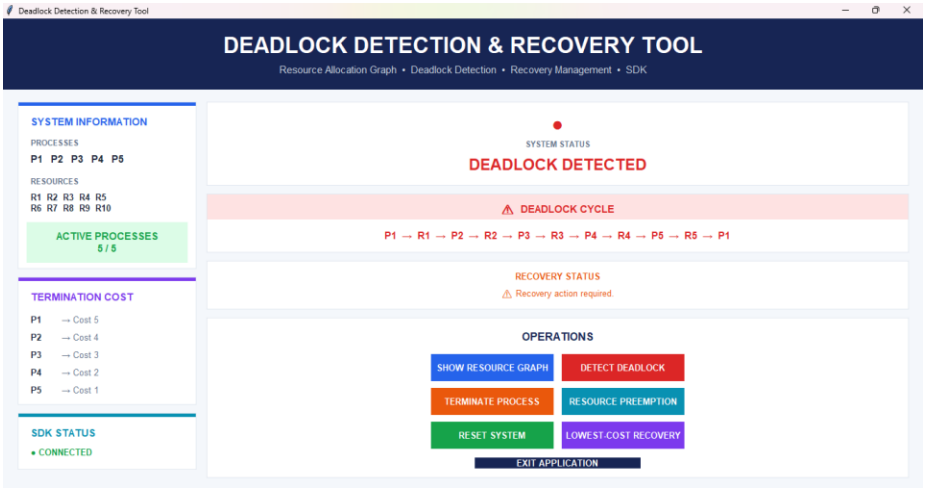

Figure 4: Clicking "Detect Deadlock" changes the system status to DEADLOCK DETECTED.

After the user clicks Detect Deadlock, the DFS-based detector runs over the default graph and confirms a cycle. The system status card turns red and reads "DEADLOCK DETECTED", the Deadlock Cycle panel prints the exact cycle P1 → R1 → P2 → R2 → P3 → R3 → P4 → R4 → P5 → R5 → P1, and the Recovery Status panel prompts that a recovery action is required. This matches exactly what the algorithm in Section 12.1 is expected to produce.

Step 2 — Process Termination (P1)

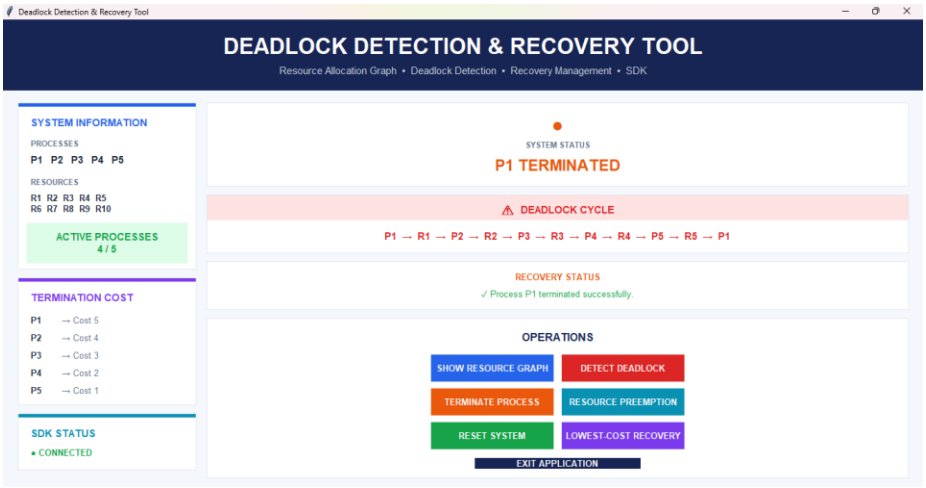

Figure 5: Terminating P1 removes it from the graph and updates the active process count to 4/5.

Choosing Terminate Process and selecting P1 removes that process node and its edges from the graph. The Active Processes counter drops to 4/5, the system status changes to "P1 TERMINATED", and the Recovery Status panel confirms "Process P1 terminated successfully." Because the deadlock cycle display is not re-drawn until the next detection, the previous cycle is

still shown for reference; running Detect Deadlock again would confirm whether the cycle involving P1 has actually been broken.

Step 2— (continued) Resource Allocation Graph - Process Termination (P1):

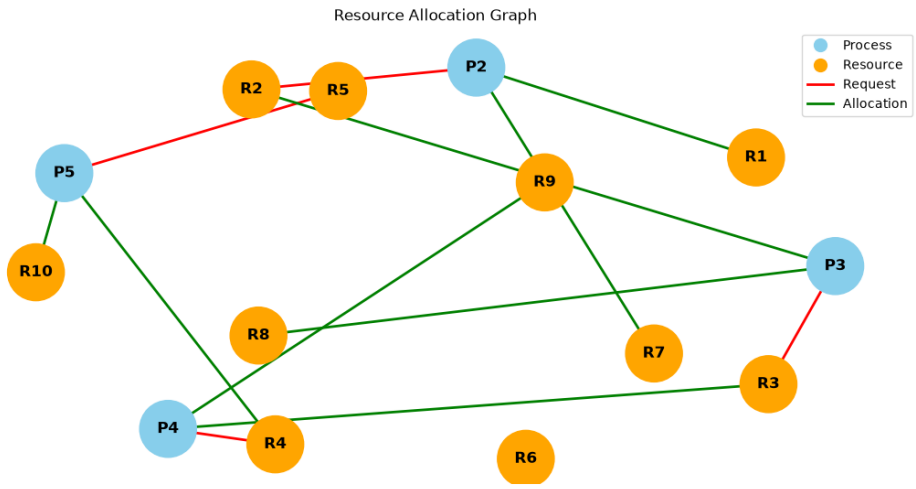

Figure 6: The Resource Allocation Graph, shown again Terminating P1 removes it from the graph

Step 3 Resource Preemption (R2: P3 → P2) —

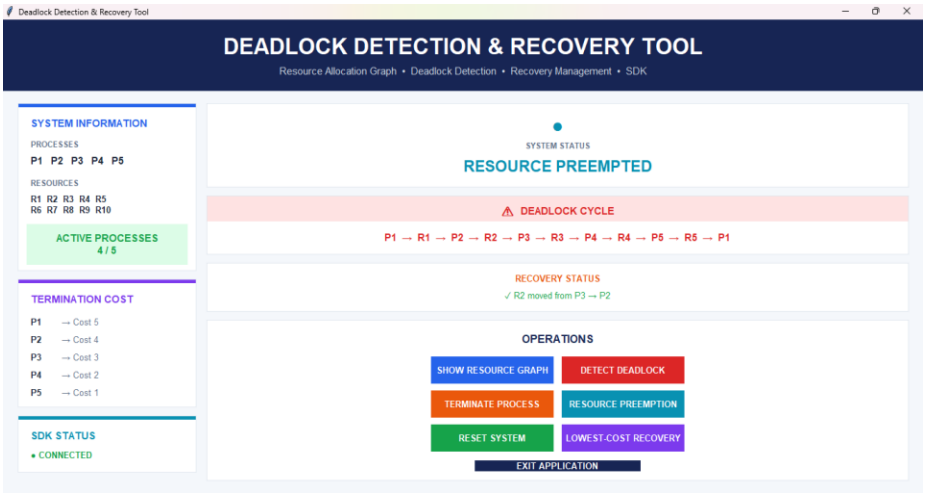

Figure 7: Resource Preemption reallocates R2 from P3 to P2, shown as "R2 moved from P3 → P2."

Selecting Resource Preemption and specifying resource R2, holder P3, and requester P2 removes the old allocation edge (R2 → P3) and the old request edge (P2 → R2), then creates a new allocation edge (R2 → P2). The system status changes to "RESOURCE PREEMPTED" and the

Recovery Status panel confirms “R2 moved from P3 → P2”, exactly as implemented in DeadlockRecovery.recover_by_preemption() (Section 12.3.2).

Step 3 (continued) — Resource Preemption (R2: P3 → P2) RAG Graph:

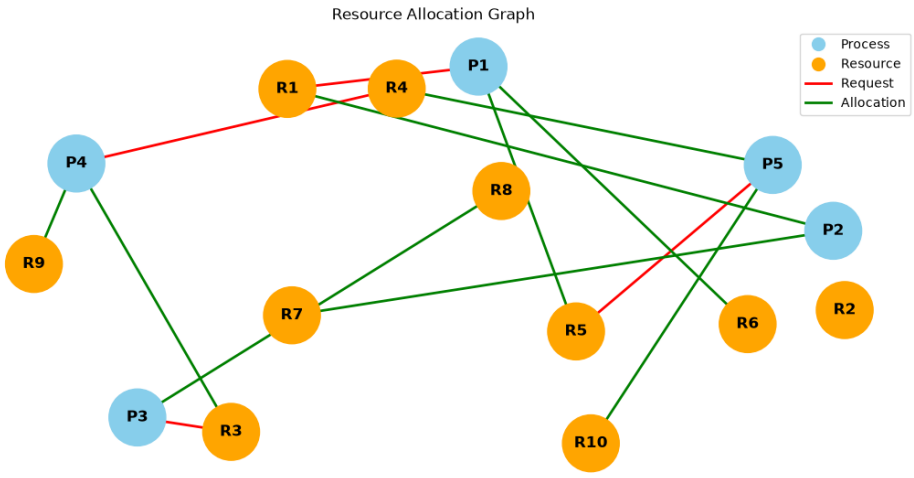

Figure 8: Resource Preemption reallocates R2 from P3 to P2, shown as “R2 moved from P3 → P2” Graph.

Step 4 — System Reset (Confirmation)

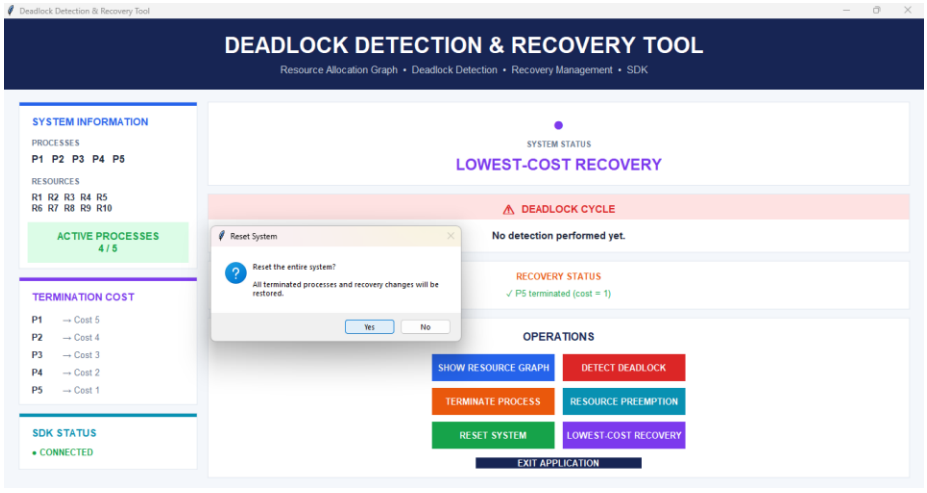

Figure 9: The Reset System action asks for confirmation before restoring the default scenario.

Clicking Reset System opens a confirmation dialog warning that all terminated processes and recovery changes will be restored to the original scenario. This safeguard prevents an accidental reset from discarding an in-progress demonstration.

Step 4 (continued) — Reset Successful

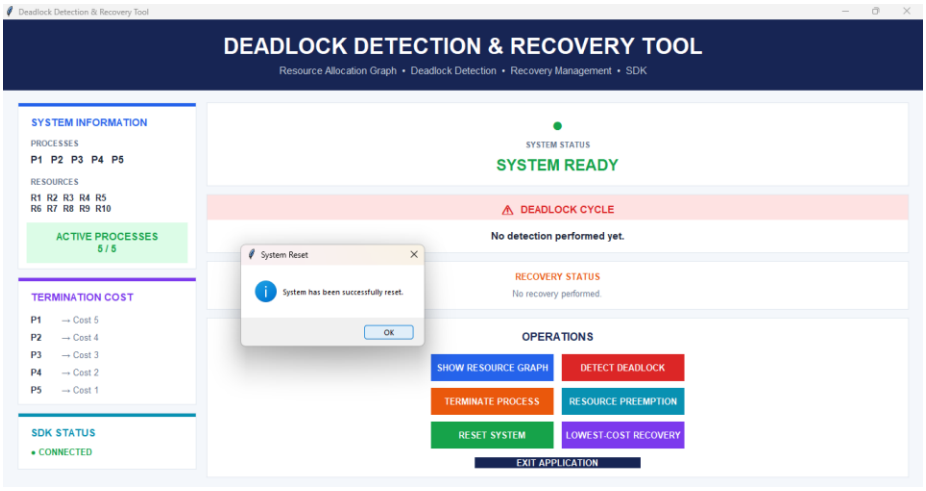

Figure 10: Confirming the reset restores all five processes and returns the system status to SYSTEM READY.

After confirming with Yes, the tool rebuilds the original Resource Allocation Graph from scratch — all five processes are active again (5/5), and the system status returns to the green "SYSTEM READY" state, with both the deadlock-cycle and recovery-status panels cleared.

Step 5 — Lowest-Cost Recovery (Confirmation)

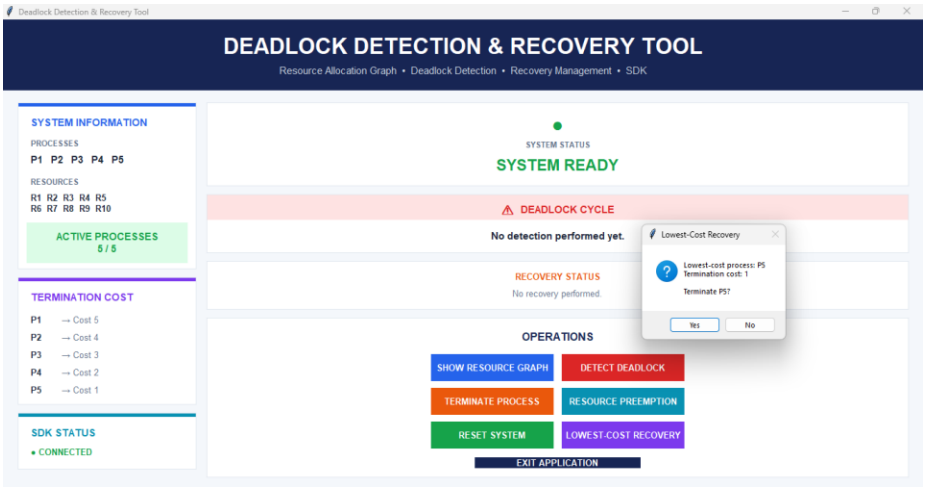

Figure 11: The Lowest-Cost Recovery dialog identifies P5 as the cheapest process to terminate (cost = 1) and asks for confirmation.

Clicking Lowest-Cost Recovery triggers the cost comparison described in Section 12.3.3. Since P5 has the lowest configured termination cost (1), the tool proposes terminating P5 and asks the user to confirm with "Terminate P5?" before proceeding.

Step 5 (continued) — P5 Terminated Successfully

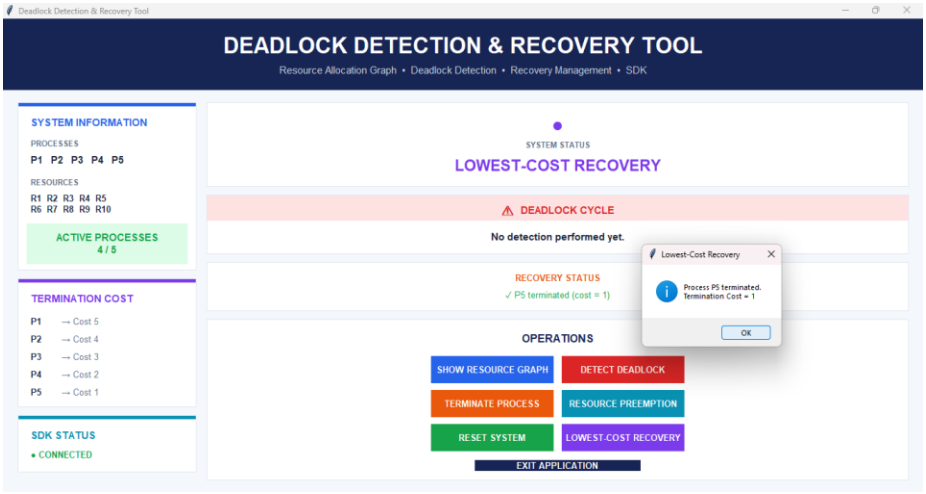

Figure 12: The confirmation dialog reports that P5 was terminated with termination cost 1.

After confirming, P5 is terminated using the same routine as ordinary process termination. A confirmation dialog reports “Process P5 terminated. Termination Cost = 1”, the Active Processes counter drops to 4/5, and the system status changes to “LOWEST-COST RECOVERY”, with the Recovery Status panel showing “P5 terminated (cost = 1).”

Final State — Resource Allocation Graph after P5 Terminated:

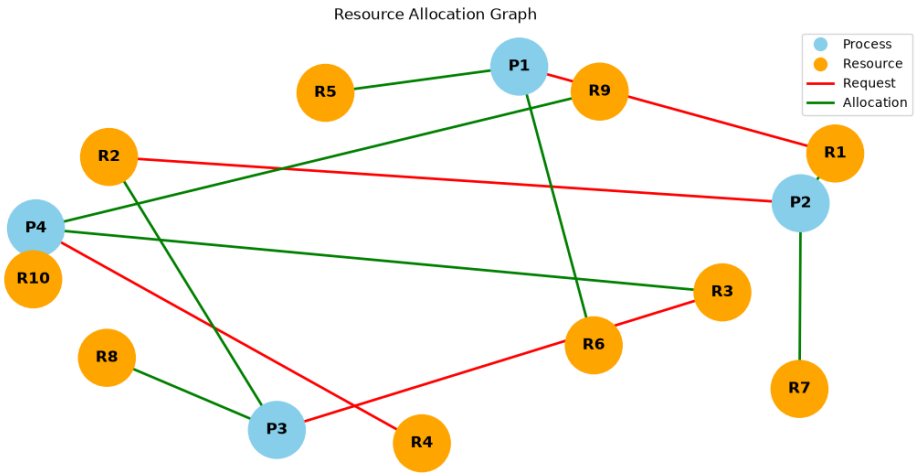

Figure 13: The Resource Allocation Graph shown again after the P5 Terminated

Reopening Show Resource Graph at the end of the walkthrough displays the same process/resource/request/allocation model used throughout the demonstration, confirming that the visualization stays consistent with the underlying graph object at every stage of the detect-and-recover cycle.

16. Testing & Sample Verification

The project was verified through functional execution of the core modules and the GUI workflow, covering graph construction, detection, all three recovery methods, system reset, SDK-level access, and GUI integration.

Test ID	Test	Expected / Observed Result	Status
T01	Graph creation	Processes/resources added and request/allocation relationships created.	**Pass**
T02	Deadlock detection	DFS identifies the configured 10-edge cycle.	**Pass**
T03	Cycle reporting	Exact cycle is returned and displayed.	**Pass**
T04	Process termination	Selected process node is removed.	**Pass**
T05	Resource preemption	Valid holder/requester/resource relationship is checked and reassigned.	**Pass**
T06	Lowest-cost recovery	P5 is selected initially because cost = 1.	**Pass**
T07	System reset	Original scenario is rebuilt.	**Pass**
T08	SDK detection	SDK returns deadlock status and cycle.	**Pass**
T09	SDK lowest-cost lookup	SDK returns P5 and cost 1 initially.	**Pass**
T10	GUI integration	Dashboard launches and exposes core operations.	**Pass**

16.1 Sample Verification

A representative run against the default scenario produced the output below, confirming that the detector, cycle reporter, and lowest-cost lookup all behave as designed. The corresponding GUI state for the detection step (T02/T03) is shown for visual confirmation.

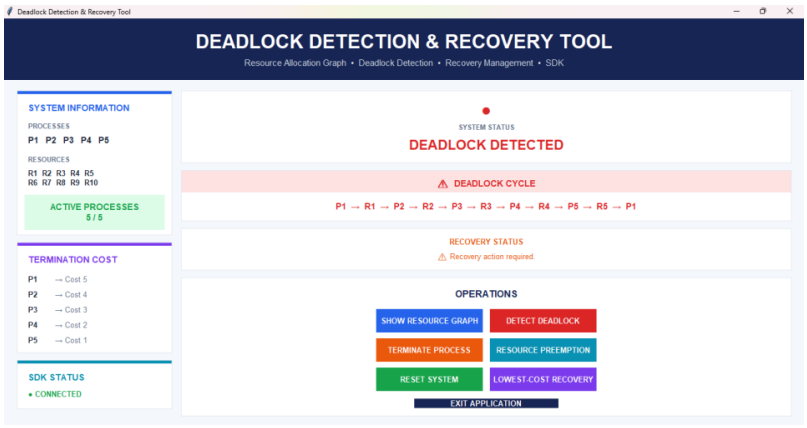

Figure 14: GUI evidence for T02/T03 — the detector correctly reports the deadlock and the exact cycle.

```
System Status: DEADLOCK DETECTED
Cycle: P1 → R1 → P2 → R2 → P3 → R3 → P4 → R4 → P5 → R5 → P1
Lowest-Cost Process: P5
Termination Cost: 1
```

17. Limitations

- Single-Instance Resource Model: The current implementation models each resource as a single instance; multi-instance resource detection (as required for the general Banker's-Algorithm-style detection) is not implemented.
- Fixed Demo Data: The default scenario uses a fixed set of 5 processes and 10 resources; there is no dynamic form yet for an end user to freely add an arbitrary number of processes/resources through the GUI at runtime.
- Static Termination Costs: Process termination costs used for lowest-cost recovery are hard-coded rather than derived from real system metrics (CPU time consumed, priority, remaining work, etc.).
- No Persistence: The tool does not save or load graph states between sessions; every run starts from the same reset scenario.
- Single-Machine Scope: The detection algorithm is designed for a single, centralized graph and does not address distributed deadlock detection across multiple machines or nodes.

18. Future Work

18.1 Short-Term Extensions

- Add a GUI form for freely creating custom processes, resources, and edges instead of relying only on the fixed demo scenario.

- Add basic input validation and error messaging for malformed preemption/termination requests entered through dialogs.

18.2 Medium-Term Extensions

- Support multi-instance resources and extend the detector to a matrix-based (Banker's-style) detection algorithm.
- Add a step-by-step DFS animation so learners can watch the recursion stack grow and the cycle close in real time.
- Persist graph scenarios to file so a session can be saved, reloaded, and shared.
- Derive termination costs from configurable, realistic metrics rather than static numbers.

18.3 Long-Term Extensions

- Extend the SDK to a small REST API so the detection/recovery engine can back a web or mobile client.
- Explore distributed deadlock detection across multiple cooperating processes/nodes.
- Add automated regression tests (e.g., pytest) covering the scenarios currently verified manually in Section 16.

19. Literature Review

Deadlock detection using resource allocation graphs and their variants has been studied extensively in operating-systems literature. The following sources were reviewed while designing the detection and recovery logic used in this project.

19.1 Graph-Theoretic Deadlock Detection (IEEE)

Ni, Sun, and Ma's peer-reviewed paper models the Resource Allocation Graph using adjacency and path matrices and detects deadlock by identifying strongly-connected components within the directed graph.

Their experiments show that the strongly-connected-component approach can effectively identify exactly which resources and processes are trapped in a cycle, which directly parallels this project's own approach of returning the full deadlock cycle (not merely a yes/no answer) once a back-edge is found during DFS.

Source: https://doi.org/10.1109/IAS.2009.64

19.2 Deadlock Detection Algorithms (GeeksforGeeks)

This reference explains that when every resource type has a single instance, checking for a cycle in the Resource Allocation Graph is a sufficient condition for deadlock, while systems with multiple instances of a resource type require an extended, matrix-based detection algorithm similar in spirit to the Banker's Algorithm's safety check.

This distinction directly informed one of the acknowledged limitations of the current project (Section 17): the tool currently implements the single-instance, cycle-based case rather than the multi-instance matrix-based case.

Source: https://www.geeksforgeeks.org/operating-systems/deadlock-detection-algorithm-in-operating-system/

19.3 Deadlock Detection and Recovery Strategies (GeeksforGeeks)

This reference lays out the standard recovery strategies once a deadlock has been detected: aborting one or all deadlocked processes, or preempting resources from one process to give to another, and notes that a detection algorithm typically needs to be re-run after each recovery step to confirm the deadlock has actually been resolved.

This project implements both strategies described in that reference — process termination and resource preemption — plus an additional cost-aware variant (lowest-cost recovery), and re-runs detection after every recovery action exactly as recommended.

Source: https://www.geeksforgeeks.org/operating-systems/deadlock-detection-recovery/

19.4 Summary of Findings

Across the reviewed literature, two consistent themes emerge: (1) cycle detection in a Resource Allocation Graph is the standard and most intuitive way to detect deadlock when resources are single-instance, and (2) recovery is most commonly achieved through process termination or resource preemption, ideally guided by some notion of cost to minimize the impact of recovery. Both themes are directly reflected in the design of this project's DFS-based detector and its three recovery strategies.

20. Conclusion

The Deadlock Detection & Recovery Tool brings together a graphical Resource Allocation Graph, a DFS-based cycle-detection algorithm, three recovery strategies, and a reusable SDK layer into a single, modular application. By separating the GUI, the SDK, and the core logic into distinct layers, the project makes it possible to test and reuse the detection and recovery engine independently of the interface that happens to be driving it.

The project meets its stated objectives: deadlock is detected exactly and visually, recovery can be carried out through more than one method, the lowest-cost strategy demonstrates cost-aware recovery, and the SDK layer confirms that the core engine does not depend on Tkinter to function. The modular file structure (graph, detection, recovery, SDK, GUI) also makes the codebase straightforward to extend with the future improvements outlined in Section 18.

21. References

- Silberschatz, A., Galvin, P. B., & Gagne, G. Operating System Concepts. Wiley.
- Tanenbaum, A. S., & Bos, H. Modern Operating Systems. Pearson.
- Ni, Q., Sun, W., & Ma, S. (2009). Deadlock Detection Based on Resource Allocation Graph. IEEE IAS 2009.
- GeeksforGeeks. Deadlock Detection Algorithm in Operating System.
- GeeksforGeeks. Deadlock Detection and Recovery.
- NetworkX Developers. NetworkX Documentation — Graph data structures and algorithms.
- Matplotlib Development Team. Matplotlib Documentation.
- Python Software Foundation. Python Documentation.
- TkDocs. Tkinter / Tk documentation and GUI programming references.

22. GitHub Repository

GitHub: https://github.com/FRFarhana/Deadlock-Detection-Recovery-Tool

QR Code Scanner:

Appendix A: Project Structure

```
Deadlock-Detection-Recovery-Tool/
├── graph.py
├── deadlock_detection.py
├── deadlock_recovery.py
├── sdk.py
├── gui.py
├── main.py
└── __pycache__/   (generated Python cache files)
```

File	Role
graph.py	ResourceAllocationGraph class and graph visualization.
deadlock_detection.py	DeadlockDetector class and DFS cycle detection.
deadlock_recovery.py	DeadlockRecovery class for termination and preemption.
sdk.py	DeadlockSDK reusable interface.
gui.py	Tkinter GUI and user interaction.
main.py	Command-line demonstration and recovery selection.

Appendix B: Key Implementation Snippets

B.1 RAG Request and Allocation

```
rag.request_resource("P1", "R1")
rag.allocate_resource("R1", "P2")
```

B.2 DFS Cycle Detection (core loop)

```
for neighbor in self.graph.successors(node):
    if neighbor not in visited:
        cycle = self._dfs(neighbor, visited, recursion_stack, path)
        if cycle:
            return cycle
    elif neighbor in recursion_stack:
        start_index = path.index(neighbor)
        return path[start_index:] + [neighbor]
```